

Five new mixed van der Waerden numbers, with machine-checked upper bounds

Leo Y. Zhang*

Independent researcher, United Kingdom
academic.submissions26@gmail.com

September 2026

Abstract

We establish five previously unknown mixed van der Waerden numbers, and establish them to a standard at which no bound rests on a solver’s unsupported verdict. Every upper bound is a SAT refutation obtained by cube-and-conquer and reduced to formally checked proof objects: 23,851 per-cube DRAT proofs in total, each replayed to `s VERIFIED` by `drat-trim` against a single formula, and for each family a checked proof that the cube set is exhaustive. In every certified refutation the symmetry breaking is switched off, so what is certified is the raw encoding rather than a reduced one. Each refutation was additionally re-derived in a configuration carrying no reversal-symmetry constraint, which agreed in every case. Every lower bound ships as an explicit colouring, checkable against the definition in milliseconds by a standalone program that uses only the Python standard library and never invokes a SAT solver.

The five values are $w(14; 2^{12}, 3, 4) = 57$, $w(21; 2^{19}, 3, 3) = 52$, $w(9; 2^7, 4, 4) = 68$, $w(6; 2^4, 4, 5) = 84$ and $w(11; 2^9, 3, 5) = 74$. Each extends by one term a sequence in the On-Line Encyclopedia of Integer Sequences (A217058, A217005, A217007, A217236, A217059) whose published values, due to Ahmed, had stood since 2012 or 2013.

Reaching full certification for the two families whose two targets are equal required removing a colour-symmetry rule from the cube generator: it discards colour-permutation images, which is sound when searching but is not a refutation, and the exhaustiveness check correctly refuses to close without it. For every family, the validation gate that reproduces the published adjacent term was run to completion before the extension was claimed. All code, certificates, proof-object tooling and result records are public.

2020 Mathematics Subject Classification: 05D10; 68R05.

1 Introduction

Van der Waerden’s theorem [15] states that for any positive integers $t_1, \dots, t_r$ there is a least n , the van der Waerden number $w(r; t_1, \dots, t_r)$, such that every r -colouring of $\{1, \dots, n\}$ contains, for some i , a t_i -term arithmetic progression entirely in colour i . Exact values are notoriously scarce; see Landman and Robertson [13] for background. The classical diagonal case $w(2; 6, 6) = 1132$ was settled by Kouril and Paul [12], and SAT-based methods have been the dominant tool for exact van der Waerden computations since Dransfield et al. [8] demonstrated their reach; see also Ahmed, Kullmann and Snevily [5] for the off-diagonal family $w(2; 3, t)$.

*Corresponding author. ORCID: <https://orcid.org/0009-0008-1145-7969>

This paper concerns the *mixed* numbers $w(j + r; 2^j, t_1, \dots, t_r)$, in which j of the colour classes have target 2. A class with target 2 can hold at most one element, since any two integers form a 2-term progression, so these classes behave as a budget of j *wildcards*: positions excused from the problem at a cost of one budget unit each. Ahmed computed extensive tables of such numbers [1, 2, 3, 4] and entered several families into the On-Line Encyclopedia of Integer Sequences in 2012 [14], adding one further term to A217059 in March 2013. The five families treated here — A217058 = $w(j + 2; 2^j, 3, 4)$, A217005 = $w(j + 2; 2^j, 3, 3)$, A217007 = $w(j + 2; 2^j, 4, 4)$, A217236 = $w(j + 2; 2^j, 4, 5)$ and A217059 = $w(j + 2; 2^j, 3, 5)$ — had received no new terms since then.

1.1 Results

We establish the next term of each of the five families.

sequence	new term	value	status
A217058	$a(12) = w(14; 2^{12}, 3, 4)$	57	approved by the OEIS, 6 August 2026
A217005	$a(19) = w(21; 2^{19}, 3, 3)$	52	approved by the OEIS, 12 August 2026
A217007	$a(7) = w(9; 2^7, 4, 4)$	68	approved by the OEIS, 12 August 2026
A217236	$a(4) = w(6; 2^4, 4, 5)$	84	approved by the OEIS, 12 August 2026
A217059	$a(9) = w(11; 2^9, 3, 5)$	74	approved by the OEIS, 13 August 2026

Each family carries a validation gate: the published adjacent term is re-derived by the same machinery, as an independent check on the method in that family, before an extension is claimed. For A217059 that gate was finished after the computation rather than before it, so the value was held back until it had run; Section 4.5 gives the timings and the verdicts.

1.2 The certificate discipline

The two halves of each value have very different epistemic status, and the entire design of this computation follows from taking that seriously.

A *lower* bound asserts the existence of a finite object. We ship the object: an explicit colouring, printed in full in this paper, which anyone can check against the definition by hand or with a short standalone verifier (`vdw/verify_certificate.py` in the repository) that uses only the Python standard library and shares no code with the search. The lower bounds require no trust in anything reported here.

An *upper* bound asserts the absence of an object, and a SAT solver’s UNSAT answer is only as good as the claim that the formula given to the solver faithfully represents the problem, plus the solver itself. We address the first risk — the far likelier one — by proving, on every instance small enough to enumerate exhaustively, that the CNF accepts exactly the colourings the definition accepts, in both directions (Section 5); and the second, for all five values, by reducing each refutation to DRAT proof objects checked by an independently built verifier, with the symmetry breaking switched off so that what is certified is the raw encoding (Section 6).

We are explicit throughout about which claims carry which kind of evidence. For the upper bounds the answer is now uniform: each of the five rests on per-cube DRAT proofs replayed to `s` `VERIFIED`, together with a checked composition proof that the cube set covers the whole assignment tree. What remains trusted is `drat-trim` itself and the claim that the CNF says what the definition says, and the second of those is what the exhaustive encoding audit of Section 5 addresses.

2 Definitions and conventions

Let $t_1, \dots, t_r \geq 2$ be integers. A colouring of $\{1, \dots, n\}$ assigns each integer to one of r classes. It is *valid* for $(t_1, \dots, t_r)$ if class i contains no t_i -term arithmetic progression, for every i . The van der Waerden number $w(r; t_1, \dots, t_r)$ is the least n for which no valid colouring of $\{1, \dots, n\}$ exists; van der Waerden’s theorem guarantees it is finite.

Writing 2^j for j classes of target 2, the mixed numbers of interest are

$$w(j + r; 2^j, t_1, \dots, t_r).$$

Since a class with target 2 holds at most one element, an equivalent formulation is

$$w(j + r; 2^j, t_1, \dots, t_r) = 1 + \max\{n : \{1, \dots, n\} \text{ admits a valid colouring with at most } j \text{ wildcards}\},$$

where a colouring with wildcards assigns each integer either to one of the r real classes or to a wildcard position that belongs to no class. In certificates below, $.$ denotes a wildcard and 1, 2 the two real classes.

For a family with fixed targets we write $a(j) = w(j + 2; 2^j, t_1, t_2)$, following the OEIS entries, which index each family by the number of wildcards with offset 0.

3 Method

3.1 Encoding

For given n and j , introduce Boolean variables $v(i, c)$ for $i \in \{1, \dots, n\}$ and $c \in \{0, 1, \dots, r\}$, where $c = 0$ means “wildcard”:

- exactly one class per position: one clause $\bigvee_c v(i, c)$ and pairwise exclusions;
- no monochromatic progression: for each class c with target t_c and each t_c -term arithmetic progression $A \subseteq \{1, \dots, n\}$, the clause $\bigvee_{i \in A} \neg v(i, c)$;
- wildcard budget: a totalizer encoding [6] of $\sum_i v(i, 0) \leq j$.

The formula is satisfiable exactly when a valid colouring exists, so $a(j)$ is the least n whose formula is unsatisfiable. Solving is done through PySAT [11] with CaDiCaL 1.9.5 [7] as the CDCL back end; proof-object generation (Section 6) uses a standalone Kissat 4.0.1 [7], for reasons given there.

3.2 Monotonicity and a free lower bound

If $\{1, \dots, n\}$ admits a valid colouring, so does $\{1, \dots, m\}$ for every $m < n$: restrict the colouring; no progression is created by deleting elements, and the wildcard count cannot increase. Satisfiability is therefore monotone decreasing in n and $a(j)$ is the unique threshold. This licenses probing candidate values directly — any SAT answer raises the floor, any UNSAT answer caps the value — rather than climbing one n at a time, and it lets the two halves of a value run as independent concurrent computations.

Monotonicity in j gives a bound for nothing: $a(j + 1) \geq a(j) + 1$. A valid colouring of $\{1, \dots, a(j) - 1\}$ with at most j wildcards extends to $\{1, \dots, a(j)\}$ by making the new final position a wildcard; it uses at most $j + 1$ wildcards and creates no monochromatic progression. Whether this free bound is tight varies by family, and its role is reported in each case: for A217007 it is the lower bound (Section 4.3); for A217059 it was three short and for A217236 four short, so those witnesses are genuine search results.

3.3 Reversal symmetry

The map $i \mapsto n + 1 - i$ is an automorphism of the problem: it carries the t -term progression $a, a + d, \dots, a + (t - 1)d$ to $n + 1 - a - (t - 1)d, \dots, n + 1 - a$, again a progression with the same common difference; it fixes every class and the wildcard count. Requiring a colouring to be lexicographically no greater than its own reversal is therefore a sound lex-leader constraint, and it halves the search space.

This matters more than it might appear. The pre-existing implementation broke only the symmetry between classes *sharing* a target value. On distinct-target families such as $(3, 4)$ that rule emits no clauses at all, so every search there had previously run with no symmetry breaking whatsoever. Adding reversal symmetry measured a $1.55\times$ speedup on the $n = 45, j = 8$ refutation, on the half of the problem that consumes essentially all the time. Because this constraint is the one piece of new mathematics in the engine, the verification programme of Sections 5 and 6 is arranged so that it is never assumed by its own check.

3.4 Cube-and-conquer and the completeness of the cube set

Refutations use cube-and-conquer [9]: branch on all class assignments to the first k positions, discard prefixes that already exceed the wildcard budget or already contain a monochromatic progression, and solve the residual formulas in parallel. The same division of labour — separate branches of one search tree, solved independently — is what Ahmed used to settle $w(2; 3, 17)$ and $w(2; 3, 18)$ [2]. At $k = 4$ the five families yield 75 cubes for targets $(3, 4)$, 36 for $(3, 3)$, 40 for $(4, 4)$, 80 for $(4, 5)$ and 76 for $(3, 5)$. A measured comparison on $n = 45, j = 8$ put $k = 4$ at 34.1 s against $k = 6$ at 65.1 s, so the finer split was not used for search.

One structural rule is worth stating because it is the difference between a theorem and a retraction: UNSAT is reported only when every cube has returned an explicit verdict. A worker killed by the operating system raises an error rather than being counted as an empty branch. That failure mode was observed during development, which is why the rule is enforced in code rather than assumed.

4 The five new terms

4.1 A217058: $w(14; 2^{12}, 3, 4) = 57$

One class must avoid 3-term progressions, the other 4-term progressions. The published terms, due to Ahmed [1, 4], are

$$a(0..11) = 18, 21, 25, 29, 33, 36, 40, 42, 45, 48, 52, 55,$$

ending at $w(13; 2^{11}, 3, 4) = 55$. The OEIS entry was queried directly on 2026-07-28 and returned exactly these twelve terms with an auto-synthesised b-file, i.e. no extension had been contributed.

Theorem 1. $a(12) = w(14; 2^{12}, 3, 4) = 57$.

Lower bound. The following colouring of $\{1, \dots, 56\}$ uses exactly 12 wildcards, has no 3-term progression in class 1 and no 4-term progression in class 2, so $a(12) > 56$:

$$2.21221212.12.22211.112.2221.222.2.1..12221211212..22212$$

Class 1 occupies 16 positions, class 2 occupies 28, and 12 are wildcards. The standalone verifier checks it in milliseconds.

Upper bound. No valid colouring of $\{1, \dots, 57\}$ with at most 12 wildcards exists. The refutation was established in four configurations, varying the symmetry breaking, the solver and the cube depth:

encoding	symmetry	solver	cubes	verdict	time
vdw4	reversal on	CaDiCaL 1.9.5	$k = 4$	UNSAT	6257.1 s
vdw2	none at all	CaDiCaL 1.9.5	$k = 4$	UNSAT	8036.6 s
vdw4	reversal off	CaDiCaL 3.0.0	$k = 5$	(not completed)	—
vdw4	reversal off	CaDiCaL 1.9.5	$k = 4$	UNSAT	8317.4 s

The second row is the one that matters: **vdw2**, the earlier engine from which **vdw4** was derived, imposes no lex-leader constraint on this family and searches the full unreduced space, so it cannot inherit an error from the reversal-symmetry constraint. It agrees. It shares its encoding core with **vdw4** and is therefore not an independent implementation; what it isolates is that one constraint, and nothing further. The third row was started and stopped after 7.4 hours without finishing; it is reported as incomplete rather than omitted. The same reversal-off question was settled in the configuration that suits this family (row four), and a second **vdw2** refutation on that pass agrees in 7259.8 s. Separately, a randomised-restart portfolio — 8 rounds $\times$ 5 seeds $\times$ 3,000,000 conflicts, with initial phases drawn from the class distribution of real witnesses — hunted for a witness at $n = 57$ and found none. Section 6 reduces this refutation to checked proof objects.

Remark 1. $a(12) - a(11) = 2$, while the published differences are 3, 4, 4, 4, 3, 4, 2, 3, 3, 4, 3, so a naive extrapolation predicts 58 or 59; both were refuted before 57 was reached. A step of 2 already occurs at $a(6) = 40 \rightarrow a(7) = 42$. The disagreement with extrapolation was treated as grounds for additional verification rather than as a curiosity.

4.2 A217005: $w(21; 2^{19}, 3, 3) = 52$

Both classes must avoid 3-term progressions. Published terms (offset 0, last extended by Ahmed in December 2012, confirmed live against the OEIS API on 2026-07-29):

$$a(0..18) = 9, 14, 17, 20, 21, 24, 25, 28, 31, 33, 35, 37, 39, 42, 44, 46, 48, 50, 51.$$

Theorem 2. $a(19) = w(21; 2^{19}, 3, 3) = 52$.

Lower bound. This colouring of $\{1, \dots, 51\}$ uses exactly 19 wildcards and contains no 3-term progression in either class, so $a(19) > 51$:

$$\dots 11.1122.2211.1122.22 \dots 11.1122.2211.1122.22$$

It was found by search in 1520.1 s and accepted by the standalone verifier. Note also that the free bound of Section 3.2 gives $a(19) \geq 52$ independently of any solver, so the refutation at $n = 52$ is the only computational input to the value.

Upper bound. $n = 52$, $j = 19$ is unsatisfiable: 4507.0 s, all 36 cubes reporting an explicit verdict. Cross-checks: **vdw2** refuted the same instance in 10650.4 s, the primary engine with reversal symmetry switched off in 11779.0 s, and **vdw2** additionally found and verified a witness of its own at $n = 51$ (1934.0 s). This refutation is also reduced to checked proof objects: 3,347 per-cube DRAT proofs and a composition proof, all replayed to **s VERIFIED** (Section 6, Table 1).

The targets here are equal, which switches on the colour-swap symmetry breaker — a code path the first family never touches. On equal-target families the two breakers together are sound but *not* canonical: they retain two representatives per orbit rather than one, because composing two independent lex-leader constraints does not canonicalise the group they generate. That costs time and cannot cost correctness, and the audit of Section 5 confirms zero orbits are lost. The family’s full-scale gate passed before the extension was claimed: published $a(18) = 51$ was reproduced exactly — SAT at $n = 50$ with a verified witness and UNSAT at $n = 51$, both at $j = 18$, in 3469.4s.

Remark 2. The witness is strikingly structured: the segment `11.1122.2211.1122.22` appears twice, separated by a block of nine consecutive wildcards. Whether the extremal colourings of this family are genuinely periodic is not something a single example can settle, and no claim is made here; it is recorded as the sort of structure worth examining if anyone pursues the family further.

4.3 A217007: $w(9; 2^7, 4, 4) = 68$

Both classes must avoid 4-term progressions. Published terms (offset 0, OEIS keyword **hard**, confirmed live against the OEIS API on 2026-07-29):

$$a(0..6) = 35, 40, 53, 54, 56, 66, 67.$$

Theorem 3. $a(7) = w(9; 2^7, 4, 4) = 68$.

Lower bound. This colouring of $\{1, \dots, 67\}$ uses exactly 7 wildcards and contains no 4-term progression in either class, so $a(7) > 67$:

`..1112112111.2221221222.1112112111.2221221222.1112112111.2221221222`

A qualification: this certificate *is* the free construction of Section 3.2 — the family gate’s $a(6)$ colouring with one further wildcard prepended. The solver, given $n = 67$, $j = 7$ cold, rediscovered that construction rather than finding a structurally different witness. It is reported as a verified confirmation, not as independent evidence, and the entire computational weight of the theorem sits on the refutation at $n = 68$.

Upper bound. $n = 68$, $j = 7$ is unsatisfiable: 7269.0s, all 40 cubes reporting an explicit verdict. Cross-checks: **vdw2**, which on this equal-target family breaks only the colour swap and carries no reversal-symmetry constraint, refuted the instance in 7375.6s, and the primary engine with reversal symmetry switched off refuted it in 7285.2s. It is also reduced to checked proof objects: 5,730 per-cube DRAT proofs and a composition proof, all replayed to **s VERIFIED** (Table 1). Being an equal-target family, this is one of the two that the colour-symmetry rule of Section 6 blocked until it was removed from the cube generator. The family gate passed before the extension was claimed: published $a(6) = 67$ was reproduced exactly — SAT at $n = 66$ with a verified witness using all 6 wildcards and UNSAT at $n = 67$, both at $j = 6$, 1032.1s for the pair.

Remark 3. The step $a(6) = 67 \rightarrow a(7) = 68$ is $+1$, the smallest possible; $+1$ already occurs twice in the published sequence, at $a(2) = 53 \rightarrow a(3) = 54$ and at $a(5) = 66 \rightarrow a(6) = 67$. The witness is conspicuously periodic — the block `1112112111.2221221222` repeats three times, and the two classes are exact complements under $1 \leftrightarrow 2$ — but since the certificate is inherited from the $a(6)$ gate rather than found independently, this says something about that colouring, not about extremal colourings of the family in general.

4.4 A217236: $w(6; 2^4, 4, 5) = 84$

Class 1 must avoid 4-term progressions, class 2 must avoid 5-term progressions. Published terms (offset 0, OEIS keyword **hard**, contributed by Ahmed in September 2012 and confirmed live against the OEIS API on 2026-07-29):

$$a(0..3) = 55, 71, 75, 79.$$

Four terms is the shortest published list of the families treated here and $j = 4$ the smallest wildcard budget, but n is the largest: the refuted instance is $\{1, \dots, 84\}$.

Theorem 4. $a(4) = w(6; 2^4, 4, 5) = 84$.

Lower bound. This colouring of $\{1, \dots, 83\}$ uses exactly 4 wildcards, with no 4-term progression in class 1 and no 5-term progression in class 2, so $a(4) > 83$:

122121221221212221.212121221121222211121.221212222.2222.212211211122221211122212122

Unlike the previous family's, this lower bound was earned rather than inherited: the free construction from $a(3) = 79$ gives only $a(4) \geq 80$, four short of the truth. The witness above was found by search (6469.7s, cube 52 of 80) and is an independent object — witnesses at $n = 80, 81, 82$ were found in an earlier pass, and the $n = 83$ colouring extends none of them, differing from all three already at the second position.

Upper bound. $n = 84, j = 4$ is unsatisfiable: 7965.0s, all 80 cubes reporting an explicit verdict. The value was bracketed rather than climbed: a ceiling probe at $n = 87$ refuted in 6688.4s, capping the value; $n = 83$ SAT raised the floor to 84; $n = 85$ UNSAT (9061.5s) dropped the ceiling to 85; and $n = 84$ UNSAT fixed the term. Monotonicity (Section 3.2) is what licenses this. It is also reduced to checked proof objects: 5,486 per-cube DRAT proofs and a composition proof, all replayed to **s VERIFIED** (Table 1). This is the one family whose exhaustiveness walk relies substantially on unit propagation rather than on single falsified clauses, for the reason given in Section 6. The refutation was established three ways:

encoding	symmetry	solver	cubes	verdict	time
vdw4	reversal on	CaDiCaL 1.9.5	$k = 4$	UNSAT	7965.0s
vdw2	none at all	CaDiCaL 1.9.5	$k = 4$	UNSAT	10959.3s
vdw4	reversal off	CaDiCaL 1.9.5	$k = 4$	UNSAT	16975.3s

vdw2 also re-ran the satisfiable side at $n = 83$ from scratch and returned a verified witness of its own in 9892.7s; it arrived at the same colouring as above. The family gate passed before the extension was attempted: published $a(3) = 79$ reproduced exactly, SAT at $n = 78$ (404.9s) and UNSAT at $n = 79$ (377.7s), both at $j = 3$.

The targets 4 and 5 are distinct, so as in Section 4.1 the colour-swap breaker emits no clauses and reversal symmetry is the only breaking in force — precisely the constraint the second and third rows above isolate. When this term was first written up, the exhaustive encoding audit did *not* cover the target pair (4, 5); rather than rest on the argument that the clause generator is generic in the targets, the audit was extended to (4, 5) on 2026-07-31 and passes: the CNF accepts exactly the colourings the definition accepts (538 and 7631 on the two audit instances, equal in both directions) and symmetry breaking loses no orbit.

Remark 4. The step $a(3) = 79 \rightarrow a(4) = 84$ is +5 against published differences 16, 4, 4, so an extrapolation from the last two steps would have said 83 — which the witness at $n = 83$ refutes outright.

4.5 A217059: $w(11; 2^9, 3, 5) = 74$

Class 1 must avoid 3-term progressions, class 2 must avoid 5-term progressions. Published terms (offset 0, OEIS keyword **hard**, confirmed live against the OEIS API on 2026-07-29):

$$a(0..8) = 22, 32, 43, 44, 50, 55, 61, 65, 70.$$

This value has a history the other four do not, which is why it appears last. It was computed and cross-checked in July alongside the others, but the family gate — reproducing the published $a(8) = 70$ as an independent check on the method in this family — had been started and killed without a verdict, and our discipline requires that gate to run to completion before an extension is claimed. The value was therefore not claimed: it was marked withheld in the public repository and kept out of the OEIS until the gate had finished. The gate was run to completion on 2026-08-11 and passed: published $a(8) = 70$ reproduced exactly — SAT at $n = 69$ with a verified witness using all 8 wildcards and UNSAT at $n = 70$, both at $j = 8$, 978.8s for the pair. A verification step that was defined and then not finished does not get waived retroactively because the answer looks right; the value is stated here because the gate ran, not because the answer stopped being in doubt — by the evidence below it never was.

Theorem 5. $a(9) = w(11; 2^9, 3, 5) = 74$.

Lower bound. This colouring of $\{1, \dots, 73\}$ uses exactly 9 wildcards, with no 3-term progression in class 1 and no 5-term progression in class 2, so $a(9) > 73$:

21121222212222.22221122112..2.2222.2222.2122..2211221212222.2222121221122

Class 1 occupies 17 positions, class 2 occupies 47, and 9 are wildcards. The bound was earned rather than inherited: the free construction of Section 3.2 applied to $a(8) = 70$ gives only $a(9) \geq 71$, three short of the truth. Witnesses were found by search and verified at $n = 71, 72$ and 73 in turn (1505.6s, 974.4s, 3464.0s). A SAT answer to a probe launched hoping for a refutation is easy to discard as a failed run; it is not — each one raises the floor, and three of them did most of the work of locating this term.

Upper bound. $n = 74, j = 9$ is unsatisfiable: 3859.9s, all 76 cubes reporting an explicit verdict. As in the previous family the value was bracketed rather than climbed: a probe at $n = 75$ refuted in 3484.7s, capping the value, while the witness at $n = 73$ raised the floor to 74, leaving $n = 74$ the single undecided instance. It is also reduced to checked proof objects: 4,801 per-cube DRAT proofs and a composition proof, all replayed to **s VERIFIED** (Table 1). The refutation was established three ways:

encoding	symmetry	solver	cubes	verdict	time
vdw4	reversal on	CaDiCaL 1.9.5	$k = 4$	UNSAT	3859.9s
vdw2	none at all	CaDiCaL 1.9.5	$k = 4$	UNSAT	3452.9s
vdw4	reversal off	CaDiCaL 1.9.5	$k = 4$	UNSAT	3460.0s

The targets are distinct, so as in Sections 4.1 and 4.4 the colour-swap breaker emits no clauses and **vdw2** searches the full unreduced space. **vdw2** also re-ran the satisfiable side at $n = 73$ from scratch and found a verified witness of its own in 2416.4s — and unlike the previous family’s, it is a *different* colouring, diverging from the one printed above at position 5, so the satisfiability of $n = 73$ is certified by two independent objects.

Remark 5. $a(9) - a(8) = 4$ against published differences 10, 11, 1, 6, 5, 6, 4, 5; a step of 4 already occurs at $a(6) = 61 \rightarrow a(7) = 65$. The differences of this family are erratic — an 11 and a 1 sit adjacent — which is precisely why the free bound was a poor predictor and the value had to be cornered rather than extrapolated.

5 Verification of the encoding and engine

The upper bounds are only as strong as the claim that the formula given to the solver represents the problem. Five independent guards were applied; all tooling named here is in the public repository.

5.1 The encoding equals the definition

On instances small enough to enumerate exhaustively, the set of colourings satisfying the CNF was compared against the set accepted by a direct transcription of the definition that never inspects a clause. They agreed exactly, in both directions, on all 11 cases tested, spanning target shapes (3, 4), (3, 3), (3, 5), (3, 3, 3), (4, 4) and (4, 5) — one shape for every family claimed in this paper. Nothing invented, nothing lost. (`vdw/encoding-audit.py`)

5.2 Symmetry breaking loses nothing

The same audit checks that every orbit of the symmetry group — generated by the reversal and by swaps of equal-target classes — retains at least one representative. Losing an orbit is exactly how a satisfiable instance would be reported unsatisfiable. Zero orbits were lost in any case tested. On distinct-target families, including (3, 4), the breaking is additionally exactly canonical: 11,539 orbits, 11,539 survivors.

5.3 The engine reproduces what is already known

All twelve published terms of A217058 were re-derived by the same engine that produced the thirteenth: for each $a(j) = w$ this means finding a verified witness at $n = w - 1$ and refuting $n = w$. A further 29 published values across five families were reproduced with the reversal-symmetry constraint enabled. Each family's full-scale gate at the adjacent index was run to completion before its extension was *claimed*; the gates for the other four families are reported in Sections 4.2–4.5, and for A217059 that gate ran after the computation rather than before it (Section 4.5). For A217058 the gate is $a(11) = 55$ at $j = 11$: SAT at $n = 54$ with a verified witness using all 11 wildcards (1872.5 s) and UNSAT at $n = 55$ (1239.2 s). This gate matters because every other replayed value tops out at $j = 10$; a defect confined to large j would have escaped the rest of the testing. It did not exist.

5.4 The cardinality constraint at full scale

The exhaustive audit reaches only $n \leq 12$ and $j \leq 3$, while the headline instance runs at $j = 12$, and the totalizer's structure grows with the bound. Too strong a cardinality constraint would forbid legal colourings and produce precisely the kind of false refutation that matters. Tested directly at $n = 55, \dots, 58$ with $j = 12$, from both sides: 240 assignments at the limit were accepted and 240 over the limit were rejected, and forcing 13 wildcards inside the full formula is correctly unsatisfiable. (`vdw/scale_test.py`)

5.5 Cross-checks against the reversal constraint

Every established value’s refutation was re-derived through `vdw2`, the earlier engine from which `vdw4` was derived, which carries no reversal-symmetry constraint, with agreement in every case, as detailed per family above. `vdw2` shares its progression enumeration, its clause construction and its cardinality encoding with `vdw4`: it is not an independent implementation, and a defect common to both would not be caught by these runs. What they isolate is the reversal lex-leader constraint, the one component `vdw4` adds. The stronger guarantee is the one in Section 6, where symmetry breaking is switched off altogether and what is certified is the raw encoding. The records are the `crosscheck_*.json` files in the repository.

6 Formally checked proof objects for all five refutations

The guards of Section 5 target an encoding defect, which is by far the likelier failure. A defect in the solver itself is a different risk, and the standard remedy is a DRAT proof of unsatisfiability [16]: the solver logs its reasoning, and an independent checker replays the log against the formula. What remains to be trusted shrinks to the checker plus the encoding audit.

6.1 Scope of the certification

We state the perimeter plainly. The refutations behind the published rungs $a(0)$ through $a(6)$ of A217058, and the first rungs of the other four families, are replayed under `drat-trim` on every full run of the repository’s verification gate and come back **s VERIFIED**.

All *five* new upper bounds are certified by the cube-level route described below: for each, every cube’s refutation is replayed to **s VERIFIED** and the cube set is proved exhaustive by a composition proof that is itself replayed to **s VERIFIED**. Table 1 records the sizes.

sequence	value	cubes	lemmas	clauses in F'
A217058	$w(14; 2^{12}, 3, 4) = 57$	4,487	3,236	7,944
A217005	$w(21; 2^{19}, 3, 3) = 52$	3,347	3,104	6,482
A217059	$w(11; 2^9, 3, 5) = 74$	4,801	3,222	10,243
A217007	$w(9; 2^7, 4, 4) = 68$	5,730	3,331	10,171
A217236	$w(6; 2^4, 4, 5) = 84$	5,486	3,545	11,827

Table 1: Per-cube proof obligations and the composition proof that discharges exhaustiveness. Every cube and every composition tail replays to **s VERIFIED**.

Two remarks are worth making because they are easy to get wrong.

First, switching symmetry breaking off in the *formula* is not sufficient. The cube generator applied the same idea a second time, discarding a prefix whenever two colours sharing a target value appeared out of canonical order. That is sound when searching, since some other prefix is its colour-permutation image, but it is not a refutation: the discarded prefix is consistent with F , precisely because F no longer carries the symmetry-breaking clauses. The per-cube proofs therefore never covered those branches, and the exhaustiveness check fails closed on them, naming an uncovered prefix. The rule is a no-op whenever the targets are pairwise distinct, which is why it went unnoticed while only A217058, with targets 3 and 4, was certified; it is exactly the two equal-target families, A217005 and A217007, that it blocked. Removing it roughly doubles the cube set and changes nothing for the other three.

Second, **A217236** is the family here for which k exceeds j by the most (**A217007**, with $k = 8 > j = 7$, invoked the fallback exactly once), and it is the only one whose exhaustiveness walk makes substantial use of the unit-propagation fallback (351 such prefixes, against 184 refuted by a single falsified clause). This is expected rather than anomalous: a prefix spending more than j wildcards violates the cardinality bound, and that bound is a totalizer encoding, so it refutes by propagation across several clauses instead of falsifying one. A propagation conflict is as sound a refutation as a falsified clause, but the walk must retain the fallback for any instance with $k > j$.

In all certified refutations, symmetry breaking is switched off: a proof of a symmetry-broken formula does not prove the original statement, and the reversal lex-leader constraint — the one piece of new mathematics in the engine — must not be assumed by the thing meant to check it. What is certified is the raw encoding. Proofs are generated by a standalone Kissat 4.0.1, because the PySAT interface returns truncated proofs from every bundled solver (the refutation never closes), and checked by **drat-trim** [16] built from byte-identical upstream source. Only the checker’s literal **s VERIFIED** line counts as a pass; every other outcome, including a parse failure, fails the gate.

6.2 The cube-level route in detail, on $n = 57$, $j = 12$

Proof size grows about $2.4\times$ per rung in this family, which puts a single-shot proof of the headline instance at order 6–17 hours and 30–150 GB. Instead the search space was split at depth $k = 8$ into 4,487 cubes. Each cube’s refutation was produced with symmetry breaking off and replayed to **s VERIFIED** under **drat-trim**, and all 4,487 records carry the same formula SHA-256, so they refute one formula rather than several. The largest individual proof was under 200 MB.

Exhaustiveness of the cube set is proved rather than assumed (**vdw/cube_exhaustive.py**). The checker does not trust the cube generator: it walks the full ternary prefix tree, and for every prefix *not* in the cube set it exhibits the clause of the formula that refutes it — for targets (3, 4) all 662 dropped prefixes are refuted by a monochromatic-progression clause already present, and no prefix is dropped by the colour-symmetry rule, which for distinct targets emits nothing at all. The resulting 3,236 tail lemmas are themselves replayed by **drat-trim** and come back **s VERIFIED**. Hence F and $F \cup \{-C : C \in \text{cubes}\}$ have identical models, every cube is refuted, and the empty clause follows. This is the same compositional pattern used at much larger scale for the Boolean Pythagorean triples problem [10].

The pipeline is fail-closed by construction: a prefix wrongly dropped yields a tail lemma that is not RUP, which **drat-trim** rejects; a missing cube clause leaves an internal lemma underivable. Both failure modes were observed failing before any positive result was believed, alongside a proof truncated to half, a proof of a different formula, a real proof checked against a satisfiable formula, and, for each of several published rungs, the instance one below the published term — which is satisfiable and must report SAT, never **VERIFIED**, and does.

7 Limitations

These are finite, checkable extensions of tracked sequences by one term each. They are not structural results: they say nothing about the growth of the families and provide no new proof technique. The exhaustive encoding-equals-definition audit reaches only instances small enough to enumerate ($n \leq 12$, $j \leq 3$); beyond that scale its guarantee is extended by the scale tests of Section 5 and by replay of known values, not by exhaustion. The proof objects reduce what must be trusted but do not eliminate it: **drat-trim** and the encoding-equals-definition claim both remain inside the trusted base. All computations ran on a single 8-core desktop machine; per-instance solver times are reported so that the scale of the computation is not overstated.

8 Declarations

Use of generative AI

The author declares the following use of a generative AI system in the preparation of this work: Anthropic’s Claude, used through the Claude Code command-line interface. The project ran from July to September 2026 and the underlying model changed over that period as successive models of Anthropic’s Opus series were released, most recently Claude Opus 5; the series and the period are given rather than a single version number, which would misstate a project of this length.

Software and computation. The SAT encodings, the verification and proof-checking tooling, the DRAT pipeline and the test suites were written with substantial assistance from that system, and the computations were run and supervised with it. This is use of an AI system as a programming tool, on the same footing as the SAT solver and the proof checker on which this paper also depends. The resulting code is public, and its outputs are checked by the independent means described below rather than accepted on the authority of whatever produced them.

Text. The manuscript was drafted and revised with assistance from the same system. Every sentence in it has been reviewed, edited and approved by the author.

Mathematics and accountability. The research questions, the choice of families and targets, the design of the verification programme, and every claim made in this paper are the author’s. The AI system is not an author, it advances no claim of its own, and it bears no responsibility for the contents. The author takes full responsibility for the whole of this work, including those parts prepared with AI assistance.

This is stated plainly because the reader is entitled to weigh it, and because the verification programme of Sections 5 and 6 is designed precisely so that no trust in any tool, human, or model is required: every lower bound is a printed colouring checkable against the definition by hand or by a standard-library script that shares no code with the search; and every upper bound, without exception, is a set of DRAT proofs replayed by an independently built checker whose own failure modes were tested first, together with a checked proof that those cubes cover the search. Nothing in this paper asks to be believed on the strength of the process that produced it.

9 Reproducibility

All code, certificates, result records and proof-object tooling are public at <https://github.com/Leo-Y-Zhang/MathRecords>. The lower-bound certificates check in milliseconds with no solver installed:

```
python vdw/verify_certificate.py "<colouring>" <j> <t1> <t2>
```

The encoding audit (`vdw/encoding_audit.py`), the scale test (`vdw/scale_test.py`), the published-value replays (`vdw/vdw_validate.py`) and the refutations (`vdw/vdw_probe.py`) run with Python and PySAT; the repository’s `verify_all.py` gate runs the full battery, including the DRAT ladder when `kissat` and `drat-trim` binaries are present. Each family’s proof-object certification is reproduced by `vdw/cube_certify.py` followed by `vdw/cube_exhaustive.py`, both resumable; the recorded runs are the `vdw/cube_run_*` directories, each holding its per-cube results, the run status and the composition certificate. Times on an 8-core desktop ranged from under three hours (A217005, A217059) to about five (A217236).

One practical note, because it cost a run. Worker count is bounded by disk throughput, not by cores or memory: `kissat` peaks near 33 MB and `drat-trim` near 420 MB, but A217236 writes proofs of 200–650 MB each, and fourteen concurrent workers writing and re-reading those made

every cube fail. Six workers verified the same cubes cleanly, and twelve measured twice as slow as six on identical work.

Build notes for the two binaries, including two silent Windows-specific pitfalls that produce wrong results rather than errors, are in `vdw/DRAT.md`.

Disclosure statement

The author reports that there are no competing interests to declare.

Data availability statement

All code, certificates, proof-object tooling and result records supporting this paper are openly available at <https://github.com/Leo-Y-Zhang/MathRecords>.

Funding

This work received no external funding.

References

- [1] T. Ahmed, *Some new van der Waerden numbers and some van der Waerden-type numbers*, *Integers* **9** (2009), A06, 65–76.
- [2] T. Ahmed, *Two new van der Waerden numbers: $w(2; 3, 17)$ and $w(2; 3, 18)$* , *Integers* **10** (2010), #A32, 369–377.
- [3] T. Ahmed, *On computation of exact van der Waerden numbers*, *Integers* **11** (2011), #A71.
- [4] T. Ahmed, *Some more van der Waerden numbers*, *Journal of Integer Sequences* **16** (2013), Article 13.4.4.
- [5] T. Ahmed, O. Kullmann and H. Snevily, *On the van der Waerden numbers $w(2; 3, t)$* , *Discrete Applied Mathematics* **174** (2014), 27–51.
- [6] O. Bailleux and Y. Boufkhad, *Efficient CNF encoding of Boolean cardinality constraints*, in: *Principles and Practice of Constraint Programming (CP 2003)*, *Lecture Notes in Computer Science* **2833**, Springer, 2003, 108–122.
- [7] A. Biere, K. Fazekas, M. Fleury and M. Heisinger, *CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling entering the SAT Competition 2020*, in: *Proc. of SAT Competition 2020 – Solver and Benchmark Descriptions*, Department of Computer Science Report Series B, vol. B-2020-1, University of Helsinki, 2020, 50–53.
- [8] M. R. Dransfield, L. Liu, V. W. Marek and M. Truszczyński, *Satisfiability and computing van der Waerden numbers*, *The Electronic Journal of Combinatorics* **11**(1) (2004), #R41.
- [9] M. J. H. Heule, O. Kullmann, S. Wieringa and A. Biere, *Cube and conquer: guiding CDCL SAT solvers by lookaheads*, in: *Hardware and Software: Verification and Testing (HVC 2011)*, *Lecture Notes in Computer Science* **7261**, Springer, 2012, 50–65.

- [10] M. J. H. Heule, O. Kullmann and V. W. Marek, *Solving and verifying the Boolean Pythagorean triples problem via cube-and-conquer*, in: Theory and Applications of Satisfiability Testing (SAT 2016), Lecture Notes in Computer Science **9710**, Springer, 2016, 228–245.
- [11] A. Ignatiev, A. Morgado and J. Marques-Silva, *PySAT: a Python toolkit for prototyping with SAT oracles*, in: Theory and Applications of Satisfiability Testing (SAT 2018), Lecture Notes in Computer Science **10929**, Springer, 2018, 428–437.
- [12] M. Kouril and J. L. Paul, *The van der Waerden number $W(2,6)$ is 1132*, Experimental Mathematics **17** (2008), no. 1, 53–61.
- [13] B. M. Landman and A. Robertson, *Ramsey Theory on the Integers*, 2nd edition, Student Mathematical Library **73**, American Mathematical Society, 2014.
- [14] OEIS Foundation Inc., *The On-Line Encyclopedia of Integer Sequences*, <https://oeis.org>, 2026. Sequences A217005, A217007, A217058, A217059, A217236.
- [15] B. L. van der Waerden, *Beweis einer Baudetschen Vermutung*, Nieuw Archief voor Wiskunde **15** (1927), 212–216.
- [16] N. Wetzler, M. J. H. Heule and W. A. Hunt, Jr., *DRAT-trim: efficient checking and trimming using expressive clausal proofs*, in: Theory and Applications of Satisfiability Testing (SAT 2014), Lecture Notes in Computer Science **8561**, Springer, 2014, 422–429. Tool at <https://github.com/marijnheule/drat-trim>.